

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Курсовой проект по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: С. А. Сорокин
Группа: М8О-306Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Курсовой проект 24/25

Задача: Дан набор многоугольников на плоскости и набор точек. Для каждой точки определите, в каком многоугольнике она лежит. Решение должно работать online, то есть должно обрабатывать запросы по одному после построения необходимой структуры данных по входным данным. Чтение входных данных и запросов вместе и построение по ним общей структуры запрещено.

Формат ввода

В первой строке даны два числа n и m ($1 \leq n, m \leq 10^5$) — количество многоугольников и количество точек соответственно. В следующих n строках заданы многоугольники. Строки вводятся в таком виде: $k \ x_1 \ y_1 \dots x_k \ y_k$, где k - количество точек в многоугольнике, x, y ($-10^9 \leq x, y \leq 10^9$) - координаты точек. В следующих m строках даны пары чисел x, y - координаты точек.

Формат вывода

Для каждого запроса выведите номер многоугольника, внутри которого содержится точка (многоугольники нумеруются с нуля), либо -1.

1 Описание

Для решения данной задачи я использовал персистентное Декартово дерево. Для реализации Декартова дерева я решил использовать метод клонирования путей, так как для решения будет достаточно частичной персистентности. Пользуясь информацией из [1], мы убеждаемся, что все операции в нем будут выполняться за $\Theta(\log n)$, где n - количество вершин. Сложность по памяти не будет превышать $\Theta(n * \log n)$.

Для начала проведём через каждую вершину всех многоугольников вертикальную прямую вниз, то есть проекцию на ось X . Получим полосы. Пусть каждой полосе соответствует точка, через которую проведён левый край полосы. Будем хранить отсортированный вектор x -координат, тогда за $\Theta(\log n)$ можно найти, в какой полосе лежит наша точка, подающаяся на проверку.

В каждой полосе на разной высоте лежат отрезки - ребра многоугольников, которые могут пересекаться только в концах по построению. В конкретной версии дерева мы будем хранить ребра многоугольников, которые пересекают соответствующую полосу. Тогда за $\Theta(2 * \log n)$ можно найти ребро, которое лежит над точкой, и ребро, которое лежит под точкой. Если эти ребра принадлежат одному многоугольнику, значит точка лежит внутри него.

2 Исходный код

Мой код можно логически поделить на две части. Сначала я реализую персистентное Декартово дерево.

```
1 struct Point {
2     int x;
3     int y;
4     Point* prev;
5     Point* next;
6     int number_polygon;
7 };
8 struct PointToArr {
9     int x;
10    Point* point;
11 };
12 struct Node {
13     int priority;
14     Point* point_left;
15     Point* point_right;
16     Node* left;
17     Node* right;
18     Node(int pr, Point* ptr_left, Point* ptr_right) : priority(pr), point_left(ptr_left
19     ), point_right(ptr_right), left(nullptr), right(nullptr) {}
20 };
21
```

Важным элементом в коде дерева является реализация персистентности во вставке и удалении.

```
1 Node* insert(Node* root, Point* point_left, Point* point_right, int priority) {
2     if (!root)
3         return new Node(priority, point_left, point_right);
4     Node* newNode = new Node(*root);
5     float k = (root->point_left->y - root->point_right->y) / (root->point_left->x -
6     root->point_right->x);
7     float b = root->point_left->y - k * root->point_left->x;
8     float left_line_y = k * point_left->x + b;
9     if (point_left->y < left_line_y || ((point_left->y == left_line_y)&&(point_right->y
10     < root->point_right->y))) {
11         newNode->left = insert(root->left, point_left, point_right, priority);
12         if (newNode->left && newNode->left->priority > newNode->priority)
13             newNode = rotateRight(newNode);
14     } else {
15         newNode->right = insert(root->right, point_left, point_right, priority);
16         if (newNode->right && newNode->right->priority > newNode->priority)
17             newNode = rotateLeft(newNode);
18     }
19     return newNode;
20 }
```

Во второй части кода в функции *main* я реализую правильное заполнение вектора точек и его сортировку, создание всех версий дерева, поиск нужной версии дерева для введенной точки и вывод ответа. Ниже приведен фрагмент кода, отвечающий за заполнение вектора точек, и одновременно за правильное связывание соседних вершин многоугольников:

```

1  std::vector<PointToArr> all_points;
2  for (int i = 0; i < n; i++){
3      for (int j = 0; j < matrix_polygons[i].size(); j++){
4          int prev = j - 1;
5          int next = j + 1;
6          if (prev < 0)
7              prev += matrix_polygons[i].size();
8          if (next == matrix_polygons[i].size())
9              next -= matrix_polygons[i].size();
10         matrix_polygons[i][j].prev = &matrix_polygons[i][prev];
11         matrix_polygons[i][j].next = &matrix_polygons[i][next];
12         PointToArr point_to_arr;
13         point_to_arr.x = matrix_polygons[i][j].x;
14         point_to_arr.point = &matrix_polygons[i][j];
15         all_points.push_back(point_to_arr);
16     }
17 }
18 std::sort(all_points.begin(), all_points.end(), compareByX);

```

kp.cpp	
Node* rotateRight(Node* y)	Правый поворот дерева
Node* rotateLeft(Node* x)	Левый поворот дерева
Node* rotateRightCopy(Node* y)	Правый поворот с копированием узла
Node* rotateLeftCopy(Node* x)	Левый поворот с копированием узла
Node* insert(Node* root, int value, int priority)	Вставка в персистентное дерево
Node* remove(Node* root, int value)	Удаление из персистентного дерева
int upper_segment(Point* point, Node* root, int closest_above=-2)	Функция поиска ребра, который выше точки
int lower_segment(Point* point, Node* root, int closest_above = -2)	Функция поиска ребра, который ниже точки
bool compareByX(const PointToArr& a, const PointToArr& b)	Компаратор для сортировки вектора точек
int binary_search(const std::vector<PointToArr>& all_points, int target_x)	Функция бинарного поиска нужной версии дерева

3 Консоль

```
dmitrykochev$ cat test1.txt
3 5
4 1 1 2 5 5 5 6 1
3 7 3 9 7 10 3
3 3 7 5 11 7 7
3 2
9 4
5 8
8 4
4 4
dmitrykochev$ ./kp <test1.txt
Number of polygon: 0
Number of polygon: 1
Number of polygon: 2
Number of polygon: 1
Number of polygon: 0
dmitrykochev$ cat test2.txt
2 4
3 7 3 9 7 10 3
3 3 7 5 11 7 7
1 1
3 7
8 5
12 1
dmitrykochev$ ./kp <test2.txt
-1
Number of polygon: 1
Number of polygon: 0
-1
```

4 Тест производительности

В тесте производительности я замеряю, сколько по времени работает моя программа, а также сколько памяти она занимает при разном количестве данных. Первый тест будет содержать 10^3 вершин во всех многоугольниках и 10^3 точек для проверки. Вторым содержит 10^4 вершин, 10^4 точек. Третий - 10^5 вершин, 10^5 точек. Также сделаем усложненный тест, в котором будут 10^5 вершин, 10^5 точек, но все многоугольники будут располагаться друг над другом.

```
dmitrykochev$ ./benchmark <test1.txt
Algorithm time: 7834us
Algorithm use: 229904 bytes
dmitrykochev$ ./benchmark <test2.txt
Algorithm time: 66914us
Algorithm use: 2023968 bytes
dmitrykochev$ ./benchmark <test3.txt
Algorithm time: 735499us
Algorithm use: 21163264 bytes
dmitrykochev$ ./benchmark <test_hard.txt
Algorithm time: 615134us
Algorithm use: 201709136 bytes
```

Последний тест занимает примерно 0.7 секунды и 20 мегабайт. Усложненный тест занимает 0.6 секунды и 190 мегабайт. Результаты можно назвать успешным. Тест производительности доказывает эффективность моего алгоритма.

5 Выводы

Выполнив курсовой проект по предмету «Дискретный анализ», я научился программировать пространственный поиск. Задача научила меня правильно использовать персистентные структуры данных, мыслить широко и на несколько шагов вперед. Безусловно, данный курсовой проект дал мне ценнейший опыт в написании и оптимизации кода. Мне пришлось продумывать реализацию и работу сложной структуры данных, а так же замерять количество памяти, которую она использует. Все это, определенно, поможет мне в решении больших задач в будущем.

Список литературы

- [1] *Персистентные структуры данных*
URL: https://neerc.ifmo.ru/wiki/index.php?title=Персистентные_структуры_данных (дата обращения: 11.11.2024).
- [2] *Локализация в ППЛГ методом полос*
URL: [https://neerc.ifmo.ru/wiki/index.php?title=Локализация_в_ППЛГ_методом_полос_\(персистентные_деревья\)](https://neerc.ifmo.ru/wiki/index.php?title=Локализация_в_ППЛГ_методом_полос_(персистентные_деревья))
(дата обращения: 15.12.2024).